

```
// COOS PRACTICAL EXAM MICRO PRINT - EXP 2, 4, 5, 6, 8, 9, 10, 11
```

```
// 24CE1045 Aaryan Choube | Batch A/A1 | 2-Column Print (7.0 pt)
```

```
// [EXP 2] BOOTH'S MULTIPLICATION (SIGNED 2'S COMPLEMENT)
```

```
public class BoothsAlgorithm {
    static void add(int[] a, int[] m, int n) {
        int c = 0;
        for (int i = n - 1; i >= 0; i--) {
            int s = a[i] + m[i] + c;
            a[i] = s % 2; c = s / 2;
        }
    }
    static void asr(int[] a, int[] q, int[] q_1, int n) {
        q_1[0] = q[n - 1];
        for (int i = n - 1; i > 0; i--) q[i] = q[i - 1];
        q[0] = a[n - 1];
        int msb = a[0];
        for (int i = n - 1; i > 0; i--) a[i] = a[i - 1];
        a[0] = msb;
    }
    public static void main(String[] args) {
        int n = 4, q_1[] = {0}, a[] = new int[4];
        int m[] = {0, 1, 1, 1}, q[] = {0, 0, 1, 1}; // 7 * 3
        int m_neg[] = {1, 0, 0, 1}; // -7 (2's comp)
        for (int i = 0; i < n; i++) {
            if (q[n-1] == 1 && q_1[0] == 0) add(a, m_neg, n);
            else if (q[n-1] == 0 && q_1[0] == 1) add(a, m, n);
            asr(a, q, q_1, n);
        }
        System.out.print("Product: ");
        for (int x : a) System.out.print(x);
        for (int x : q) System.out.print(x);
        System.out.println(" (21)");
    }
}

// [EXP 2] RESTORING DIVISION (UNSIGNED DIVIDEND / DIVISOR)
public class RestoringDivision {
    public static void main(String[] args) {
        int n = 4, q[] = {1, 0, 1, 1}, m[] = {0, 0, 1, 1}; // 11 / 3
        int a[] = new int[5], m_neg[] = {1, 1, 1, 0, 1}; // -3 (5-bit)
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < 4; j++) a[j] = a[j + 1];
            a[4] = q[0];
            for (int j = 0; j < 3; j++) q[j] = q[j + 1];
            int c = 0, temp[] = new int[5];
            for (int j = 4; j >= 0; j--) {
                int s = a[j] + m_neg[j] + c;
                temp[j] = s % 2; c = s / 2;
            }
            if (temp[0] == 1) q[3] = 0; // Negative -> Restore (keep a)
            else { a = temp; q[3] = 1; }
        }
        int quot = 0, rem = 0;
        for (int x : q) quot = quot * 2 + x;
        for (int x : a) rem = rem * 2 + x;
        System.out.println("Quotient=" + quot + ", Remainder=" + rem);
    }
}

// [EXP 2] NON-RESTORING DIVISION ALGORITHM
public class NonRestoringDivision {
    public static void main(String[] args) {
        int n = 4, q[] = {1, 0, 1, 1}, m[] = {0, 0, 0, 1, 1}; // 11 / 3
        int a[] = new int[5], m_neg[] = {1, 1, 1, 0, 1};
        for (int i = 0; i < n; i++) {
            int isNeg = a[0];
            for (int j = 0; j < 4; j++) a[j] = a[j + 1];
            a[4] = q[0];
            for (int j = 0; j < 3; j++) q[j] = q[j + 1];
            int[] addend = (isNeg == 1) ? m : m_neg;
            int c = 0;
            for (int j = 4; j >= 0; j--) {
                int s = a[j] + addend[j] + c;
                a[j] = s % 2; c = s / 2;
            }
            q[3] = (a[0] == 0) ? 1 : 0;
        }
        if (a[0] == 1) { // End correction
            int c = 0;
            for (int j = 4; j >= 0; j--) {
                int s = a[j] + m[j] + c;
                a[j] = s % 2; c = s / 2;
            }
        }
        int quot = 0, rem = 0;
        for (int x : q) quot = quot * 2 + x;
        for (int x : a) rem = rem * 2 + x;
        System.out.println("Quotient=" + quot + ", Remainder=" + rem);
    }
}
```

```
// [EXP 4] LINUX SHELL SCRIPT (SYSTEM INFORMATION - A,B,C,D,E)
```

```
#!/bin/bash
echo "[a] OS: $(uname -o), Kernel: $(uname -r) $(uname -v)"
echo "[b] Top 10 CPU Processes:"
ps -eo pid,user,%cpu,comm --sort=-%cpu | head -n 11
echo "[c] Top 10 Memory Processes:"
ps -eo pid,user,%mem,comm --sort=-%mem | head -n 11
echo "[d] User: $(whoami), Logname: $(logname 2>/dev/null || whoami)"
echo "[e] Shell: $SHELL, Home: $HOME, OS: $OSTYPE"
echo "      Dir: $(pwd), PATH: $PATH"
```

```
// [EXP 5] LINUX KERNEL APIS (CP, LS, MV IN C)
```

```
// my_cp.c (open, read, write, close)
#include <fcntl.h>
#include <unistd.h>
int main(int c, char *v[]) {
    int src = open(v[1], O_RDONLY);
    int dst = open(v[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    char buf[1024]; int r;
    while ((r = read(src, buf, 1024)) > 0) write(dst, buf, r);
    close(src); close(dst);
    return 0;
}
// my_ls.c (opendir, readdir, closedir)
#include <dirent.h>
#include <stdio.h>
int main() {
    DIR *d = opendir("."); struct dirent *e;
    while ((e = readdir(d)) != NULL)
        if (e->d_name[0] != '.') printf("%s\n", e->d_name);
    closedir(d); return 0;
}
```

```
// my_mv.c (rename system call)
```

```
#include <stdio.h>
int main(int c, char *v[]) { return rename(v[1], v[2]); }
```

```
// [EXP 6] PROCESS CREATION (FORK, ORPHAN, ZOMBIE)
```

```
// fork_demo.c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main() {
    pid_t p = fork();
    if (p == 0) { // Child
        printf("Child PID=%d, Parent PPID=%d\n", getpid(), getppid());
    } else if (p > 0) { // Parent
        wait(NULL); // Reaps child, avoids zombie
        printf("Parent PID=%d, Child=%d reaped\n", getpid(), p);
    }
    return 0;
}
// Orphan: Child sleeps 5s, Parent exits. Child PPID becomes 1 (systemd).
// Zombie: Child exits(0), Parent sleeps without wait(). Child state is Z.
```

```
// [EXP 8] BANKER'S DEADLOCK AVOIDANCE ALGORITHM
```

```
import java.util.Arrays;
public class BankersAlgorithm {
    public static void main(String[] args) {
        int P = 5, R = 3;
        int[][] alloc = {{0,1,0},{2,0,0},{3,0,2},{2,1,1},{0,0,2}};
        int[][] max = {{7,5,3},{3,2,2},{9,0,2},{2,2,2},{4,3,3}};
        int[] avail = {3, 3, 2};
        int[][] need = new int[P][R];
        for (int i = 0; i < P; i++)
            for (int j = 0; j < R; j++) need[i][j] = max[i][j] - alloc[i][j];
        boolean[] finish = new boolean[P];
        int[] seq = new int[P], work = avail.clone();
        int count = 0;
        while (count < P) {
            boolean found = false;
            for (int p = 0; p < P; p++) {
                if (!finish[p]) {
                    int j;
                    for (j = 0; j < R; j++) if (need[p][j] > work[j]) break;
                    if (j == R) {
                        for (int k = 0; k < R; k++) work[k] += alloc[p][k];
                        seq[count++] = p; finish[p] = true; found = true;
                    }
                }
            }
            if (!found) { System.out.println("Unsafe!"); return; }
        }
        System.out.println("Safe Seq: " + Arrays.toString(seq));
    }
}
```

```
// [EXP 9] DYNAMIC MEMORY MANAGEMENT (FIRST, BEST, WORST FIT)
```

```
public class MemoryManagement {
    static void allocate(int[] bOrig, int[] p, String type) {
        int[] b = bOrig.clone(), alloc = new int[p.length];
        for (int i = 0; i < p.length; i++) {
            int idx = -1;
            for (int j = 0; j < b.length; j++) {
```

```

        if (b[j] >= p[i]) {
            if (type.equals("FIRST")) { idx = j; break; }
            if (type.equals("BEST") && (idx == -1 || b[j] < b[idx])) idx =
j;
            if (type.equals("WORST") && (idx == -1 || b[j] > b[idx])) idx =
j;
        }
        alloc[i] = idx;
        if (idx != -1) b[idx] -= p[i];
    }
    System.out.print(type + ": ");
    for (int x : alloc) System.out.print((x != -1 ? (x+1) : "None") + "
");
    System.out.println();
}
public static void main(String[] args) {
    int[] blocks = {100, 500, 200, 300, 600};
    int[] procs = {212, 417, 112, 426};
    allocate(blocks, procs, "FIRST");
    allocate(blocks, procs, "BEST");
    allocate(blocks, procs, "WORST");
}
}

```

// [EXP 10] PAGE REPLACEMENT (FIFO, LRU, OPTIMAL)

```

import java.util.*;
public class PageReplacement {
    static int fifo(int[] pg, int f) {
        int[] fr = new int[f]; Arrays.fill(fr, -1);
        int faults = 0, idx = 0;
        for (int p : pg) {
            boolean hit = false;
            for (int x : fr) if (x == p) { hit = true; break; }
            if (!hit) { fr[idx] = p; idx = (idx + 1) % f; faults++; }
        }
        return faults;
    }
    static int lru(int[] pg, int f) {
        int[] fr = new int[f], time = new int[f];
        Arrays.fill(fr, -1); int faults = 0;
        for (int i = 0; i < pg.length; i++) {
            boolean hit = false;
            for (int j = 0; j < f; j++) {
                if (fr[j] == pg[i]) { hit = true; time[j] = i + 1; break; }
            }
            if (!hit) {
                int minIdx = 0;
                for (int j = 1; j < f; j++) if (time[j] < time[minIdx]) minIdx =
j;
                fr[minIdx] = pg[i]; time[minIdx] = i + 1; faults++;
            }
        }
        return faults;
    }
    public static void main(String[] args) {
        int[] pages = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7,
0, 1};
        System.out.println("FIFO Faults (3 frames): " + fifo(pages, 3));
        System.out.println("LRU Faults (3 frames): " + lru(pages, 3));
    }
}

```

// [EXP 11] DISK SCHEDULING (FCFS, SSTF, SCAN, C-SCAN)

```

import java.util.*;
public class DiskScheduling {
    static int fcfs(int[] req, int head) {
        int seek = 0, cur = head;
        for (int r : req) { seek += Math.abs(r - cur); cur = r; }
        return seek;
    }
    static int scan(int[] req, int head, int diskSize) {
        int seek = 0, cur = head;
        int[] sorted = req.clone(); Arrays.sort(sorted);
        int idx = 0;
        while (idx < sorted.length && sorted[idx] < head) idx++;
        for (int i = idx; i < sorted.length; i++) { seek +=
Math.abs(sorted[i] - cur); cur = sorted[i]; }
        seek += Math.abs((diskSize - 1) - cur); cur = diskSize - 1;
        for (int i = idx - 1; i >= 0; i--) { seek += Math.abs(sorted[i] -
cur); cur = sorted[i]; }
        return seek;
    }
    public static void main(String[] args) {
        int[] req = {82, 170, 43, 140, 24, 16, 190};
        System.out.println("FCFS Total Seek Time (Head=50): " + fcfs(req,
50));
        System.out.println("SCAN Total Seek Time (Head=50): " + scan(req, 50,
200));
    }
}

```